

# 计算机网络与代理排错全书

## 结合终端实战的零基础指南

作者：写给小白的通俗指南

(包含了从 0 到 1 的底层逻辑、日志分析、双网卡冲突与代码级解释)

2026 年 6 月 2 日

### 目录

|     |                                |    |
|-----|--------------------------------|----|
| 1   | 引言：把网络世界想象成“寄快递”               | 1  |
| 2   | 第一部分：地址的秘密 (IP 与 CIDR)         | 3  |
| 2.1 | 1. 什么是 IP 地址?                  | 3  |
| 2.2 | 2. 什么是 CIDR (比如 /16)?          | 3  |
| 3   | 第二部分：快递代收点 (Proxy 与环境变量)       | 5  |
| 3.1 | 1. 什么是 Proxy (代理)?             | 5  |
| 3.2 | 2. 系统怎么知道有代收发站? (环境变量)         | 5  |
| 4   | 第三部分：案发现场日志剖析 (为什么信寄丢了?)       | 6  |
| 4.1 | 第一幕：发出去的信，石沉大海                 | 6  |
| 4.2 | 第二幕：给快递员装上“行车记录仪”              | 6  |
| 4.3 | 第三幕：破案！死板的快递员与 CIDR 盲区         | 7  |
| 4.4 | 大结局：下达死命令，强夺控制权                | 8  |
| 5   | 第四部分：遇到问题怎么解决 (给小白的三招)         | 8  |
| 5.1 | 第一招：查监控 (永远加 -v)               | 8  |
| 5.2 | 第二招：防坑测试 (--noproxy '*')       | 8  |
| 5.3 | 第三招：终极解法，告别死板规则                | 9  |
| 6   | 第五部分：双路并行——插上网线开 WiFi，电脑傻了怎么办? | 10 |
| 6.1 | 1. 两扇门与门卫的烦恼 (什么是网卡与路由表?)      | 10 |
| 6.2 | 2. 为什么开着 WiFi 门卫就会“懵逼”?        | 10 |

|       |                                   |    |
|-------|-----------------------------------|----|
| 6.3   | 3. 终极解决办法（小白必看步骤）                 | 11 |
| 6.3.1 | Linux 极客彩蛋：为什么我的系统设置里找不到「有线网络」？   | 11 |
| 7     | 第六部分：番外篇——为什么 TUN 模式能“降维打击”？      | 13 |
| 7.1   | 1. 为什么 TUN 能解决旧版软件的 Bug？          | 13 |
| 7.2   | 2. 什么是 Trie 树（前缀树）算法？             | 13 |
| 8     | 第六部分：DNS——为什么必须要有个“接线员”？          | 15 |
| 8.1   | 1. 互联网的超级“114 查号台”                | 15 |
| 8.2   | 2. 为什么路由器里要把 DNS 设为 192.168.10.1？ | 15 |
| 8.3   | 3. 代理（Clash）环境下的 DNS 终极魔法：防污染     | 16 |
| 9     | 附录一：进阶极客实验——手写 Python 模拟 CIDR 计算  | 17 |
| 10    | 附录二：TCP 三次握手的生活化比喻                | 18 |
| 10.1  | 第一步：SYN（你好，听得到吗？）                 | 18 |
| 10.2  | 第二步：SYN-ACK（听得到！我也准备好了）           | 18 |
| 10.3  | 第三步：ACK（好的，那我们开始吧！）               | 18 |

# 1 引言：把网络世界想象成“寄快递”

如果你没有计算机基础，不要害怕那些复杂的英文单词。整个计算机网络，其实就是一个巨大的“寄快递”和“收发信件”的系统。

概念对照表

- 你想看一个网页 → 你要给别人寄一封信，要求对方把网页数据给你寄回来。
- 数据包 (Packet) → 就是你寄出去的那封信。
- 路由器 (Router) → 就是各个城市的顺丰/京东快递中转站。
- IP 地址 (IP Address) → 就是门牌号（你的发件地址和目标的收件地址）。
- 端口号 (Port) → 某栋大楼里的具体房间号，或者是具体的收件人。
- 代理 (Proxy) → 强制设立在你家门口的“快递代收发站”。

本讲义将完全基于你电脑终端中发生的一起真实的“包裹丢失（网络不通）”惨案，带你彻底揭开计算机网络的底层秘密。

## 2 第一部分：地址的秘密（IP 与 CIDR）

要送信，首先得知道门牌号。但在计算机世界里，门牌号分为很多种类型。

### 2.1 1. 什么是 IP 地址？

IP 地址是设备在网络中的身份证号。在我们这次的案例中，出现了两个极其特殊的门牌号：

- **127.0.0.1（回环地址）**：这个地址永远代表“我自己”。就像你左手递东西给右手，根本不需要出门。电脑发往这个地址的信件，会在操作系统的内部直接“折返”，不需要真正的网卡和网线。这也是为什么即使你拔掉网线，你依然可以访问 127.0.0.1。
- **192.168.x.x（局域网 IP）**：这是你家“小区内部”的门牌号。比如你家的大模型服务地址是 192.168.5.9，你手机可能是 192.168.5.10。在小区里面（连接同一个 WiFi 或路由器）串门非常快。但是，小区外面的人（公网）是绝对找不到这个地址的。

### 2.2 2. 什么是 CIDR（比如 /16）？

在网络配置和日志中，你经常会看到类似 192.168.0.0/16 这样的写法。这里的 /16 究竟是什么意思？为什么它能把系统搞崩溃？

CIDR 的全称是 **Classless Inter-Domain Routing（无类别域间路由）**。你可以把它理解为“**邮政编码**”或者“**小区统称**”。它规定了：“只要门牌号前两段是 192.168 的，都是同一个小区的！”

**硬核知识：掩码的二进制计算**

在计算机底层，CIDR 是如何判断 192.168.5.9 是不是属于 192.168.0.0/16 的？

1. **转为二进制：**计算机把 IP 切成 32 个 0 和 1。
2. **/16 的含义：**意思是“前 16 个数字必须严格匹配，后面的不用管”。
3. 换算下来，/16 对应的是子网掩码 255.255.0.0。
4. 计算机将目标 IP 与掩码进行“按位与 (AND)”运算，发现匹配成功。它就知道：“哦，这是自己人！”

## 3 第二部分：快递代收点（Proxy 与环境变量）

### 3.1 1. 什么是 Proxy（代理）？

Proxy 在英文里是“代理人”的意思。你可以把它当成一个“**快递代收发站**”。当你没有代理时，你的电脑（快递员）直接出门送信。当有了代理（比如 Clash 软件），你的电脑会把信交给 Clash，Clash 帮你寄给远方，收到回信再交给你。

### 3.2 2. 系统怎么知道有代收发站？（环境变量）

操作系统为了告诉所有的软件“去哪里找这个代收发站”，设立了**环境变量**。这相当于在你家的大门上贴了两张显眼的告示：

第一张告示：强制代收规则 (http\_proxy)

```
export http_proxy="http://127.0.0.1:7890"
```

**大白话：**“所有寄出去的信，一律先交给 7890 号代收发站（Clash）处理！不管目的地是哪！”

第二张告示：豁免白名单 (no\_proxy)

```
export no_proxy="192.168.0.0/16"
```

**大白话：**“注意：如果要寄给小区邻居（/16 邮编范围），就自己亲自去送，\*\* 不要 \*\* 交给代收发站！”

了解了门牌号和门口的这两张告示，我们马上进入案发现场。

## 4 第三部分：案发现场日志剖析（为什么信寄丢了？）

根据你提供的真实终端日志，我们将案件分为四幕进行彻底剖析。

### 4.1 第一幕：发出去的信，石沉大海

#### 【原始日志分析】

```
Bash(curl -s http://192.168.5.9:3000/ 2>&1 | head -50)
__ (No output)
```

**【发生了什么？】**你雇佣了一个名叫“curl”的命令行快递员，对他说：“去敲局域网门牌号为 192.168.5.9 的 3000 号房间，把网页数据取回来。”结果，快递员去了，空着手回来了。终端没有任何输出内容。

### 4.2 第二幕：给快递员装上“行车记录仪”

为了找出问题，你给快递员加上了 `-v`（verbose 详细模式）参数，这相当于开启了他的行车记录仪。

#### 【原始日志分析】

```
Bash(curl -v http://192.168.5.9:3000/ 2>&1 | head -40)
* Uses proxy env variable no_proxy == 'localhost
  ,127.0.0.1,192.168.0.0/16...'
* Uses proxy env variable http_proxy == 'http://127.0.0.1:7890/'
* Trying 127.0.0.1:7890...
* Connected to (nil) (127.0.0.1) port 7890 (#0)
> GET http://192.168.5.9:3000/ HTTP/1.1
> Host: 192.168.5.9:3000
...
< HTTP/1.1 502 Bad Gateway
```

### 【日志逐行破解与底层逻辑】

- Uses proxy env variable...: 快递员出门前，确实看到了你贴在门上的两张告示（读取了环境变量）。
- Trying 127.0.0.1:7890...: 令人震惊的一幕出现了！快递员居然无视了白名单，把原本应该在内网递交的信件，死死地塞给了本地代理端口（Clash 代收发站）。这叫做“应用层的自愿流量劫持”。
- < HTTP/1.1 502 Bad Gateway: 最终收到了退件！为什么是 502？

#### 深度原理：502 错误究竟是谁报的？

很多人以为 502 是目标网站挂了。错！在这个场景下，**502 是 Clash 代收发站给你盖的退件戳**。代收发站拿着这封信，跑去国外的中转站找。可是外网根本找不到你家小区的私有地址 192.168.5.9。于是代收发站只能把信退回来，告诉你：  
\*\*502 网关错误（我没坏，但上家连不上，查无此路）\*\*。

## 4.3 第三幕：破案！死板的快递员与 CIDR 盲区

真正的悬念来了：你明明在告示里写了白名单 192.168.0.0/16，为什么信还是交给了代收发站？

### 【原始日志分析】

```
Bash(curl --version 2>&1 | head -1)
curl 7.81.0 (x86_64-pc-linux-gnu)...
```

原因出在快递员身上。你雇佣的老版快递员（curl 7.81.0）**文化程度太低**。

#### 版本代沟造成的“弱智”行为

curl 7.81.0 的底层 C 语言代码中，并没有实现 CIDR 的二进制按位运算功能。他把目标地址 192.168.5.9 和纸条上的 192.168.0.0/16 做了个“大家找不同”。他一看：“哎哟，这两个词的字母长得完全不一样啊！带有斜杠的什么鬼？”于是他死板地判定：\*\*不在白名单内，交代收发站！\*\*



## 4.4 大结局：下达死命令，强夺控制权

为了拿到数据，你使用了非常暴力的覆盖指令。

### 【原始日志分析】

```
Bash(curl -s --noproxy '*' http://192.168.5.9:3000/ )
<!doctype html>...
```

**【技术解析】**你在命令行中增加的 `--noproxy '*'` 参数，优先级是全宇宙最高的。它强制覆盖了系统门口贴的告示。你揪着快递员的耳朵下死命令：“把之前看的告示全忘了！不管纸条上写什么，你给我亲自走到 192.168.5.9 去！”这一次，快递员终于直接通过 ARP 协议在局域网内找到了目标，拿回了网页数据。

## 5 第四部分：遇到问题怎么解决（给小白的三招）

以后遇到网络连不通、代理报错，不要慌，按照以下三个层级去排查和解决。

### 5.1 第一招：查监控（永远加 `-v`）

这是每个程序员必备的条件反射。遇到 `curl` 报错，立刻加上 `-v` 参数。

```
curl -v http://你的目标网站.com
```

如果输出里有 `* Trying 127.0.0.1:7890...`，说明流量被代理劫持了。

### 5.2 第二招：防坑测试（`--noproxy '*'`）

当你怀疑代理把网络搞乱了，想证明“如果没有代理，我是不是就能连上？”，请使用这个命令：

```
curl --noproxy '*' http://你的目标网站.com
```

如果加上这句立马能连上，那么 100% 是代理配置（比如白名单失效）的问题。

### 5.3 第三招：终极解法，告别死板规则

1. **治本法（解雇笨员工）**：把系统的 `curl` 升级到 7.86.0 以上版本，新版原生支持 CIDR 解析。
2. **治标法（把话说直白）**：把白名单里的 `/16` 去掉，直接写死目标门牌号：`export no_proxy="192.168.5.9"`。
3. **降维打击法（开启 TUN 模式）**：在 Clash 中开启 TUN（虚拟网卡）模式。这就相当于修了自动传送带。开启 TUN 后，不用再贴环境变量告示，只要数据一出门就会掉进系统的虚拟地道，由 Clash 底层通过更高级的 Trie 树算法自动帮你完美分拣。

## 6 第五部分：双路并行——插上网线开 WiFi，电脑傻了怎么办？

很多同学会遇到一个极其经典的工业现场问题：“我电脑开着 WiFi（为了上网），同时插了一根网线连着工业机械臂（IP 是 192.168.5.1）。结果我必须把 WiFi 关掉，才能连上机械臂。这是为什么？怎么解决？”

要解开这个谜团，我们要引入计算机网络中极其重要的两个概念：“网卡”和“路由表”。

### 6.1 1. 两扇门与门卫的烦恼（什么是网卡与路由表？）

**网卡 (Network Interface)：**你的电脑只要连了网，就会开一扇门。连着 WiFi，就是开了一扇“无线门”；插着网线，就是开了一扇“有线门”。现在，你的电脑同时开了两扇门。

**路由表 (Routing Table)：**电脑内部有一个名叫“路由表”的门卫老大。每当你寄一封信时，门卫老大就要决定：“这封信，该从哪扇门丢出去？”

### 6.2 2. 为什么开着 WiFi 门卫就会“懵逼”？

分析你电脑底层的真实路由表：

```
$ ip addr
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> ... state UP
    inet6 fe80::d693:90ff:fe3d:c5b8/64 scope link
3: wlp0s20f3: <BROADCAST,MULTICAST,UP,LOWER_UP> ...
    inet 192.168.8.9/24 ...

$ ip route get 192.168.5.1
192.168.5.1 via 198.18.0.2 dev Meta table 2022 src 198.18.0.1
```

### 案发现场：门卫与 Clash 的连环套

通过你电脑的真实数据，我们发现了两个致命秘密：

1. **有线网卡 (enp3s0) 根本没有 IPv4 地址！** 只有一串长长的 IPv6 地址。这意味着你的电脑在“有线门”上，连个合法的门牌号都没有！
2. **流量被 Clash TUN 彻底劫持！** 当你尝试访问机械臂 (192.168.5.1) 时，路由表显示它通过 dev Meta 发送。而 Meta 正是 Clash 开启的虚拟网卡！

**为什么必须关闭 WiFi 才能连上？** 因为你的有线网卡没有配 IP，电脑认为自己和 192.168.5.x 网段“不熟”。此时只要开着 WiFi，所有的流量（包括找机械臂的信）都会无脑掉进 Clash 的虚拟地道（TUN 接口 Meta）里，被送往外网，机械臂当然不会有任何响应。当你关闭 WiFi 时，Clash 传送带和 WiFi 大门全部关闭，电脑在无路可走的情况下，才可能会尝试通过有线网卡进行最原始的广播寻找。

## 6.3 3. 终极解决办法（小白必看步骤）

要解决这个问题，你根本不需要频繁开关 WiFi，只需要给你的有线网卡 \*\* 配置一个静态 IP\*\*。

### 6.3.1 Linux 极客彩蛋：为什么我的系统设置里找不到「有线网络」？

很多 Ubuntu 用户会遇到一个神奇的现象：打开系统设置，在网络栏里 \*\* 只能看到 VPN 和代理，根本没有「有线网络」这一栏 \*\*。

分析你本机的系统配置文件：

```
$ cat /etc/network/interfaces
auto enp3s0
iface enp3s0 inet manual
```

#### 破案：网卡被其他管理员强占了！

因为你之前可能使用过拨号上网（宽带连接 pppoeconf），导致你的网卡 enp3s0 被写入了底层的 /etc/network/interfaces 配置文件中。

而在 Ubuntu 的图形网络管理器(NetworkManager)的配置中，有一行默认设定:managed=false（不托管由 interfaces 配置文件管理的网卡）。

因此，系统桌面认为这块网卡“不归我管”，所以在系统设置图形界面里直接 \*\* 把它隐藏了 \*\*！

让「有线网络」重新在系统设置里显现的步骤：

1. **解除旧锁定**：打开终端，编辑配置文件（需要管理员权限）：

```
sudo nano /etc/network/interfaces
```

把关于 `enp3s0` 的两行（或者所有宽带拨号相关的行）前面 \*\* 加上 `#` 号进行注释 \*\*（或者直接删掉它们）。

2. **开启 NetworkManager 托管**：编辑网络管理配置文件：

```
sudo nano /etc/NetworkManager/NetworkManager.conf
```

找到 `[ifupdown]` 这一栏，把下面的 `managed=false` 改为 `managed=true`。

3. **重启网络服务**：在终端输入命令重启服务：

```
sudo systemctl restart NetworkManager
```

现在，重新打开你的 Ubuntu 系统设置，你就会惊喜地发现 \*\*「有线网络」选项完美归位 \*\*！你可以直接用鼠标去点击它，手动填写静态 IP：

- **IP 地址**： 192.168.5.10
- **子网掩码**： 255.255.255.0
- **默认网关 / DNS**： 打死也不要填！留空！

### 为什么这样配就能解决？

一旦你给有线网卡配置了 192.168.5.10，电脑的路由表里就会瞬间生成一条高优先级的局部路由规则：“凡是去往 192.168.5.x 小区的信，一律直接从有线网卡（`enp3s0`）扔出去！”

这条局部规则的优先级（掩码长度 /24）远远高于 Clash 传送带的默认全局规则（/0）。从此，即便你开着 WiFi、挂着 Clash，只要你访问 192.168.5.1，门卫老大也会雷厉风行地一脚把信踢进有线网线里，直达机械臂！而你的上网流量依然会快乐地走 WiFi 门。

## 7 第六部分：番外篇——为什么 TUN 模式能“降维打击”？

你刚才亲自测试了开启 Clash 的 TUN 模式，发现原来那个死活连不上的问题瞬间消失了。这究竟是施了什么魔法？什么是“Trie 树算法”？

### 7.1 1. 为什么 TUN 能解决旧版软件的 Bug？

我们要回到之前的比喻：**不开 TUN 时（环境变量代理）**：是快递员（curl）出门前自己看门上的告示（no\_proxy）。因为快递员太笨看不懂 /16 的小区编码，所以他送错了。这就叫“应用层代理”——极其依赖软件自身的智商。

**开启 TUN 时（虚拟网卡代理）**：你把大门上的告示全撕了（清空环境变量）。快递员出门时啥也不用想，拿着信直接往前走。但是！你在大门口挖了一个坑，装了一部**隐形的传送带**（TUN 虚拟网卡）。不管快递员多笨，他一迈出门槛，信件就会自动掉进这个传送带里，被直接送到 Clash 的“地下秘密分拣中心”。

到了分拣中心，由极其聪明的 Clash 站长亲自来看这封信。Clash 站长是懂 192.168.0.0/16 这种 CIDR 编码的。他一看信封上写着 192.168.5.9，立刻大手一挥：“这是同一个小区的信，走直连通道（Direct），不用发给国外的代收发站！”于是，信件顺利送达。

#### 硬核知识：OSI 模型的降维打击

环境变量代理工作在 **OSI 第七层（应用层）**，每个应用（curl, python, 浏览器）都有自己的实现逻辑，很容易出 Bug。TUN 模式工作在 **OSI 第三层（网络层）**。它通过修改操作系统的底层路由表，强制接管了所有通过网卡发出的 IP 数据包。应用层根本不知道自己被劫持了。这就是所谓的“降维打击”。

### 7.2 2. 什么是 Trie 树（前缀树）算法？

现在，全世界有成千上万个小区编码（CIDR 规则），Clash 站长每天要处理几万封信，他为什么能在一秒钟内判断出这封信该走国内还是国外？这就靠 **Trie 树（前缀树）**。

**如果不用 Trie 树（笨办法）**：站长拿着手里的小本本，从第一页开始一行一行

对：“192.168.5.9 是 10.0.0.0 吗？不是。”“是 172.16.0.0 吗？不是。”这样找下去，电脑早就卡死了。

**如果用了 Trie 树 (字典树算法)：** Trie 树就像是查字典。站长看到门牌号的第一位是 192，他直接翻到字典的 192 这一章；下一位是 168，他顺着目录直接翻到 168 这一页；然后是 5，他一秒钟就定位到了 192.168.5.x 这个抽屉，发现上面贴着标签：**直连 (Direct)!**

### Trie 树的威力

通过 Trie 树 (前缀匹配)，无论 Clash 里装了 10 条规则还是 10 万条规则，查找的速度几乎是一模一样的！它只需要顺着 IP 地址的每一位往下找 (对于 IPv4 最多找 32 次)，这在计算机里只要零点零几毫秒。这就是 Clash 能做到“完美且无感分拣”的底层性能密码。

## 8 第六部分：DNS——为什么必须要有个“接线员”？

在刚才修改路由器的配置中，有一个名为 **DNS 服务器** 的选项。很多初学者总是把它和网关搞混。什么是 DNS (Domain Name System)? 在“寄快递”的比喻里，它扮演了什么角色？

### 8.1 1. 互联网的超级“114 查号台”

我们在上网时，输入的是 `www.baidu.com` 或者 `github.com`。但是！快递员（电脑）只认得**纯数字**的门牌号（比如 `39.156.66.10`）。你如果直接给快递员一个名字（比如“马化腾的家”），快递员是没法送信的，因为地图上根本没有画“名字”，只有“经纬度坐标”。

这时候，就需要 **DNS（超级查号台 / 电话簿）** 了。

#### DNS 的工作流程

**你**：“快递员，帮我去 `www.baidu.com` 拿个东西。”

**快递员（电脑）**：“我不认识这几个英文字母啊！等一下，我先打个电话给 114 查号台（也就是配置里的 DNS 服务器）。”

**快递员**：“喂？114 吗？请问 `www.baidu.com` 的真实物理门牌号是多少？”

**DNS 服务器（查号台）**：“稍等，我查一下小本本... 查到了，它的门牌号是 `39.156.66.10`。”

**快递员**：“收到！”（然后快递员才真正骑上车，朝着 `39.156.66.10` 绝尘而去）。

### 8.2 2. 为什么路由器里要把 DNS 设为 192.168.10.1？

在局域网（你家）里，你的 WiFi 路由器（`192.168.10.1`）不仅是掌控大门的“门卫老大（网关）”，它还 **\*\* 兼职 \*\*** 做“片区 114 查号接线员”。当你的电脑想要查询 `www.baidu.com` 时，会直接问路由器。路由器内部如果不知道，路由器会再代替你去问“国家级的总查号台（比如电信、联通的 DNS，或者 `114.114.114.114`）”。

**如果不填 DNS 会怎样？** 如果你的电脑没有配置 DNS，你打开浏览器输入 `www.baidu.com` 会瞬间报错：**无法解析服务器的 DNS 地址**。因为快递员手里拿着收件人的名字，却死活找不到能查门牌号的电话簿，最后只能原地罢工。有趣的是，如果你当时直接输入的



是数字 IP（比如 192.168.5.9 或者 39.156.66.10），那么不配 DNS 也一样能连上，因为快递员根本不需要去查电话簿了！

### 8.3 3. 代理（Clash）环境下的 DNS 终极魔法：防污染

很多时候，即使你设了正规的查号台，你依然上不去某些国外的网站（比如 Google）。这不是因为 Google 坏了，而是因为“**查号台（DNS）被人动了手脚**”。国内的查号台在遇到你查询 Google 时，会故意给你报一个 \*\* 假的门牌号 \*\*（比如告诉你去太平洋中心），这叫做 **DNS 污染**。

#### Clash 的 Fake-IP（假查号）黑科技

为了解决查号台骗人的问题，Clash 使出了一招极其不可思议的绝技：**发假驾照**。当你的电脑向查号台查询 google.com 时，Clash 瞬间把这通电话掐断！然后 Clash 自己伪装成查号台，随便给你发一个假的门牌号（比如 198.18.0.1）。快递员拿着这个假门牌号刚出门，立刻就掉进了我们前面说的 **TUN 虚拟传送带**里！

Clash 在地下室截获了这个假门牌号，笑着说：“哈哈，这假门牌号是我发的，我知道你其实想去 Google。来，这封信我亲自通过加密通道发到国外的真代理服务器，让国外的代理服务器帮你查真正的门牌号！”

这就完美避开了国内查号台的欺骗。

## 9 附录一：进阶极客实验——手写 Python 模拟 CIDR 计算

如果你觉得光听“按位与”运算不过瘾，我们可以用 10 行 Python 代码，亲自模拟一下电脑底层是如何判断门牌号是否在同一个小区的。

```
import ipaddress

# 1. 定义你的小区白名单 (CIDR)
my_neighborhood = ipaddress.ip_network("192.168.0.0/16")

# 2. 拿到这封信的目标门牌号
target_ip = ipaddress.ip_address("192.168.5.9")

# 3. 让计算机做底层比对
if target_ip in my_neighborhood:
    print(f"[{target_ip}] 是自己人！不要交给代收发站，亲自去送！")
else:
    print(f"[{target_ip}] 是外人，把它交给代收发站！")
```

cidr\_check.py

当你运行这段代码时，输出结果一定是“自己人”。而在老版本的 curl 源代码（C 语言）中，它缺乏上面这种聪明的库，只能做：

```
if (strcmp("192.168.5.9", "192.168.0.0/16") == 0) {
    // 只有长得完全一样才放行
    deliver_directly();
} else {
    send_to_proxy();
}
```

笨笨的 C 语言逻辑

这就是我们常说的：\*\*“技术架构上的降维打击”\*\*。

## 10 附录二：TCP 三次握手的生活化比喻

在快递员出发之前，还需要经过一个“确认接头”的环节，也就是传说中的 TCP 三次握手。小白听到这个词往往觉得高深莫测，其实它就像是在打电话。

### 10.1 第一步：SYN（你好，听得到吗？）

快递员（curl）到达目标地址 192.168.5.9 门外，他并没有直接把包裹扔进去。他先在门口按响了门铃，并大喊一声：“喂！3000 号房间有人吗？我有一个快递要交接，你能听到我说话吗？”这一声呼喊，在网络协议里叫做 **SYN 包**。

### 10.2 第二步：SYN-ACK（听得到！我也准备好了）

3000 号房间的主人（服务器程序）听到门铃后，走到门边回答：“听得到！我这边已经准备好接收包裹了。那你能听到我说话吗？”这句回应，在网络协议里叫做 **SYN-ACK 包**。

### 10.3 第三步：ACK（好的，那我们开始吧！）

快递员听到主人的回应，确认了对方是个活人并且能正常交流，于是最后回答一句：“好的，我也听到你了，我们正式开始交接包裹吧！”这最后一句确认，在网络协议里叫做 **ACK 包**。

#### 为什么非要三次？

很多人问，为什么不能两次？如果只有两次：快递员喊：“有人吗？”主人答：“有！”然后主人就开始傻等。万一主人回答“有”的这句话被风吹走了（网络丢包），快递员根本没听见，快递员就走了。留下主人一个人在房间里一直等，浪费了感情和时间。只有经过 **\*\* 三次 \*\*** 确认，双方才能 100% 确定对方既能听见自己，也能向自己说话。这是一条极为严谨的通信逻辑。

全书完